PROJECT- II SYNOPSIS

On

FC Pipeline: A Multi-Agent, Fine-Grained Factual Consistency Evaluation and Correction Framework for Abstractive Summarization

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

SUBMITTED BY:	
Roll No.	**26317**
Name of Student	**Lakshit**

Under the guidance of:

Prof./Dr.

JULY – DEC 2026

DRONACHARYA COLLEGE OF ENGINEERING KHENTAWAS, FARUKH NAGAR, GURUGRAM HARYANA - 122506

DRONACHARYA
College of Engineering

APPROVAL SHEET

This Project–II synopsis entitled **"FC Pipeline: A Multi-Agent, Fine-Grained Factual Consistency Evaluation and Correction Framework for Abstractive Summarization"**, submitted by

1. **Lakshit (Roll No 26317),** signature: – .....

is approved as per the requirements of the VII Semester for the award of the degree of Bache-lor of Technology (B.Tech.) in Computer Science & Engineering (Artificial Intelligence & Machine Learning) at Dronacharya College of Engineering.

Date: – .....

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Project Mentor:

Prof./Dr. Signature: –

.....

Head of Department: .....
(Prof. Yashvardhan Soni

ABSTRACT

Abstractive summarization systems built on large language models routinely produce fluent sum-maries that are not fully faithful to their source documents – a phenomenon widely referred to as *factual inconsistency* or *hallucination*. Coarse, single-score consistency metrics are unable to say *which* part of a summary is wrong or *why*, which limits their usefulness both for benchmarking and for downstream correction. This project presents **FC Pipeline**, an end-to-end framework for fine-grained factual-consistency evaluation and correction of abstractive summaries across two domains – general news and clinical/biomedical text. The framework combines two published lines of research: **ACUEVAL** (Wan et al., ACL 2024 Findings), which decomposes a summary into *Atomic Content Units* (ACUs) and verifies each unit independently against the source document, and **TreatFact** (Luo et al., ESWA 2024), which introduces PICO-aware (Population, Intervention, Comparison, Outcome) prompting for evaluating factual consistency in clinical summaries.

The system is organised as a five-stage pipeline: a *domain router* that classifies each document–summary pair as news or clinical; an *ACU generator* that performs five-shot decomposition of the summary into atomic facts; an *ACU verifier* that issues a per-unit Yes/No judgement (with PICO-aware prompting in the clinical domain) against the source document; an *ACUEVAL scorer* that aggregates per-unit verification probabilities into a single, interpretable consistency score; and an optional *iter-ative refiner* that rewrites a summary using only its incorrect atomic units, followed by *refinement metrics* that measure the corrected summary's accuracy and how much of the originally correct content was preserved (coverage). The pipeline is model-agnostic: it can run offline against a determinis-tic mock backend for development and grading, or be pointed at OpenAI's API or a locally hosted 70B-parameter model served through vLLM, without any change to the pipeline logic.

The framework was evaluated on a hand-crafted four-example sample benchmark spanning both domains and on the full 248-sample TreatFact clinical benchmark. On the sample benchmark the pipeline achieved a balanced accuracy of 100% at a decision threshold of 0.50, correctly separat-ing consistent from inconsistent summaries with zero false positives and zero false negatives. On the larger TreatFact benchmark, the pipeline processed 248 document–summary pairs and produced a mean ACUEVAL score of 0.92, with a balanced accuracy of 0.50 at the configured threshold of 0.10, highlighting the additional difficulty of clinical fact verification and motivating future work on domain-specific verifier fine-tuning. The system also emits fully reproducible HTML and JSON re-ports containing per-sample, per-ACU breakdowns, aggregate confusion-matrix statistics, and refine-ment accuracy/coverage scores, making the evaluation process auditable end to end.

Keywords: Factual Consistency, Abstractive Summarization, Atomic Content Units, Large Lan-guage Models, Clinical NLP, Hallucination Detection, Automatic Evaluation.

TABLE OF CONTENTS

List of Figures ..... **i**

Chapter 1: Introduction **1**

Chapter 2 - Literature Review **4**

Chapter 3 – Proposed System **7**

Chapter 4 – Methodology **11**

Chapter 5 – Conclusion **22**

Chapter 6 - References **24**

Appendix A – Selected Source Listings **24**

Appendix B – Extended TreatFact Evaluation Results **26**

List of Figures

1 Fig 1.1: Representative factual-consistency check performed by the pipeline 2

2 Fig 3.1: End-to-end FC Pipeline architecture (Domain Router → ACU Generator → ACU Verifier → ACUEVAL Scorer → Refiner → Refinement Metrics) 7

3 Fig 3.2: Sequence of interactions between the CLI, the pipeline orchestrator, the domain router, the LLM backend, and the report generator 10

4 Fig 4.1: Detailed flowchart of a single document–summary evaluation, including the optional refinement loop 13

5 Fig B.1: Additional prototype figure referenced during pipeline development 31

Chapter 1: Introduction

1.1Background

Large Language Models (LLMs) have become the default engine behind modern abstractive summarization: given a long source document, the model generates a short, fluent summary in its own words rather than merely extracting sentences. This fluency is also the source of a well-documented failure mode – the generated summary can assert facts, entities, locations, dosages, or outcomes that are not actually supported by, or directly contradict, the source document. In general news summa-rization this shows up as swapped names or locations; in clinical and biomedical summarization the same failure mode can misstate a drug's effect, a trial's outcome, or a patient population, which is considerably higher-stakes.
Coarse factual-consistency metrics – a single number per summary – cannot localise the error to a specific claim, and therefore cannot drive automatic correction. This motivates a *fine-grained* ap-proach: break the summary down into small, independently checkable claims, verify each one against the source, and only then aggregate into a single score. This is the design philosophy behind the **FC Pipeline** project developed for this synopsis.

1.2 Motivation

Two influential but previously separate lines of research motivate this project:

- **ACUEVAL** (Wan et al., ACL 2024 Findings) proposes decomposing a summary into Atomic Con-tent Units (ACUs) – minimal, self-contained factual statements – and verifying each one indepen-dently against the source document using a Yes/No judgement. The final consistency score is the mean of the per-unit verification probabilities, which is both interpretable and directly localises errors to specific ACUs.

- **TreatFact** (Luo et al., ESWA 2024) targets the clinical domain specifically, and shows that domain-aware, PICO-structured prompting (Population, Intervention, Comparison, Outcome) substantially improves the reliability of factual-consistency verification for medical and biomedical text, where generic news-style prompts under-perform.

No existing open-source tool combines both approaches into a single, domain-routed pipeline that also closes the loop with automatic correction. This gap is what the project sets out to fill.

1.3 Problem Statement

Given a source document D and a machine-generated summary S, the goal is to (i) determine whether S is factually consistent with D, (ii) identify precisely which claims in S are unsupported, and
(iii) where a correction is requested, produce a refined summary S' that repairs the incorrect claims while preserving as much of the originally correct content as possible – all without requiring the same evaluation logic to be duplicated separately for the news and clinical domains.

1.4 Project Objectives

1. Design and implement a domain router that automatically classifies a document–summary pair as *news* or *clinical* and selects the appropriate prompt templates.

2. Implement five-shot ACU decomposition (the ACU Generator) that converts a summary into a list of atomic factual claim

3. Implement a per-ACU verifier that issues a Yes/No verification probability for each atomic claim against the source document, using PICO-aware prompting for the clinical domain.

4. Implement the ACUEVAL scorer that aggregates per-ACU probabilities into a single consistency score and a binary consistent/inconsistent label using a tunable decision threshold.

5. Implement an iterative summary refiner that rewrites a summary using only the ACUs that failed verification, together with refinement metrics (accuracy and coverage) that quantify the quality of the correction.

6. Build benchmark loaders and downloaders for the sample, AGGREFACT, and TreatFact datasets, and a command-line interface exposing demo, evaluate, and download commands.

7. Support three interchangeable LLM backends – an offline mock backend for deterministic test-ing, OpenAI's hosted API, and a locally hosted 70B model served via vLLM – without changing pipeline logic.

8. Generate human-readable HTML and machine-readable JSON evaluation reports with per-sample, per-ACU breakdowns and aggregate metrics (balanced accuracy, confusion matrix, mean ACUE-VAL score).

1.5 Scope of the Project

The current scope covers two domains (news and clinical/biomedical) and three benchmark sources (a hand-crafted sample set, AGGREFACT for news, and TreatFact for clinical text). The pipeline is evaluated end to end using the deterministic mock backend, with the OpenAI and vLLM backends implemented and configurable but reserved for full-scale evaluation as future work (see Chapter 5 and the Future Scope discussion in Chapter 4). The project does not attempt to train a new verification model from scratch in this phase; it orchestrates and evaluates LLM-based verification and correction.

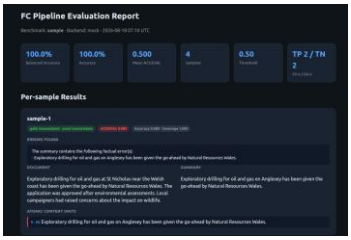


Figure 1: Fig 1.1: Representative factual-consistency check performed by the pipeline

1.6 Project Team and Individual Responsibilities

Team Member	Primary Responsibility
Lakshit	Pipeline orchestration, ACU Generator and ACU Verifier implementation, LLM backend abstraction (mock / OpenAI / vLLM), synopsis compilation.
	Domain Router, prompt-template design for the news and clinical (PICO-aware) verification paths, benchmark load-ers.
	Summary Refiner, Refinement Metrics, evaluation-metrics module (balanced accuracy, confusion matrix, threshold tuning).

1.6 Organisation of the Report

Chapter 2 reviews the literature underlying ACU-based and clinical factual-consistency evalua-tion. Chapter 3 presents the proposed system, its architecture, and its hardware/software requirements. Chapter 4 details the methodology, algorithms, flowcharts, implementation plan and schedule of work. Chapter 5 concludes the synopsis and outlines expected outcomes, and Chapter 6 lists the references consulted.

Chapter 2 – Literature Review

2.1 Evolution of Summarization Evaluation

Early automatic summarization metrics such as ROUGE and BLEU measure lexical overlap be-tween a generated summary and one or more reference summaries. These metrics correlate poorly with human judgements of *factual* correctness, because a summary can share many n-grams with a reference while still misstating a fact, and conversely can be perfectly faithful while using entirely different wording. This gap motivated a distinct line of research into *factual consistency evaluation*, which checks a summary directly against its *source document* rather than against a reference summary.

2.2 Entailment- and QA-based Consistency Metrics

Early factual-consistency metrics framed the problem as either (i) a Natural Language Inference (NLI) task, where the source document is treated as the premise and the summary (or each of its sentences) as the hypothesis, or (ii) a Question-Answering (QA) task, where questions are generated from the summary and answered from both the summary and the source; agreement between the two answers indicates consistency. Both families produce a single sentence-level or document-level score and generally use models fine-tuned for the news domain, which limits transfer to other domains such as clinical text and gives no fine-grained localisation of *which* claim is wrong.

2.3 ACUEVAL: Atomic Content Unit Evaluation (Wan et al., ACL 2024 Findings)

ACUEVAL reframes factual-consistency evaluation as a decomposition-then-verification problem. A summary is first decomposed by an LLM into a set of Atomic Content Units (ACUs) – short, self-contained factual statements, each expressing a single checkable claim. Each ACU is then verified independently against the source document with a Yes/No prompt, and the model's probability of "Yes" is taken as that unit's verification score. The document-level ACUEVAL score is the mean of the per-ACU probabilities. Because verification happens per-unit, an incorrect claim can be pinpointed exactly, in contrast to a single scalar score. This project adopts the ACUEVAL decomposition-then-verification design as the backbone of its `acu_generator` and `acu_verifier` pipeline stages (Chapter 4, Section 4.1).

2.4 TreatFact: Domain-Aware Clinical Evaluation (Luo et al., ESWA 2024)

TreatFact studies factual consistency evaluation specifically for clinical and biomedical summaries, where source documents (e.g. randomized controlled trial abstracts) describe a Population, an Inter-vention, a Comparison arm, and an Outcome (the PICO framework). Luo et al. show that generic, news-style verification prompts frequently fail on clinical text because they do not probe for the spe-cific PICO elements that determine clinical correctness – for example, confusing "no significant effect" with "significantly reduced" changes a clinical conclusion entirely, even though the two phrases are lexically similar. TreatFact instead uses PICO-aware prompts that explicitly ask the verifier to check the population, intervention, comparator, and outcome mentioned in each ACU against the source. The TreatFact benchmark (248 document–summary pairs used for evaluation in this project, Chap-ter 4, Section 4.5) is built from real clinical-trial abstracts and LLM-generated summaries, and is the standard the clinical branch of this project's Domain Router and ACU Verifier are validated against.

2.5 Benchmarks Referenced

• **AGGREFACT** (Tang et al.) – a widely used news-domain factual-consistency benchmark built from CNN/DailyMail and XSum summaries, with a "Fully-Trained SOTA" (FTSOTA) split used for evaluating recent summarization systems.
• **TreatFact** (Luo et al., 2024) – the clinical-domain benchmark described above, downloaded and converted to JSONL by the project's benchmark downloader (`fc_pipeline/data/download.py`).

• **Sample benchmark** – a hand-crafted, four-example benchmark built for this project spanning both domains, used for fast, deterministic development and grading with the mock LLM backend.

2.6 Correction / Refinement of Inconsistent Summaries

A smaller body of work looks beyond *detecting* inconsistency toward *correcting* it – rewriting a summary so that it becomes faithful to the source while retaining as much useful content as possi-ble. Correction is harder to evaluate automatically than detection, since a corrected summary must be checked both for factual accuracy *and* for coverage of the originally correct information (a trivially "safe" correction that discards all content would be factually perfect but useless). This project's Sum-mary Refiner and Refinement Metrics stages (Chapter 4, Sections 4.1 and 4.2) are designed around this two-axis evaluation: an `accuracy_score` for the corrected summary and a `coverage_score` measuring how much of the originally-correct content survives refinement.

2.7 Taxonomy of Summarization Hallucinations

Factual errors in abstractive summaries are commonly grouped into two broad categories, both of which appear in the evaluation results discussed in Chapter 4:

• **Intrinsic hallucinations** – the summary misuses information that *is* present in the source document, but distorts it (e.g. swapping two entities that both appear in the document, or reversing the direction of a reported effect). Sample-1 and sample-4 of this project's sample benchmark (Chapter 4, Section 4.6) are both intrinsic hallucinations.

• **Extrinsic hallucinations** – the summary introduces information that cannot be found anywhere in the source document at all (e.g. an invented statistic, an invented date, or an invented outcome). Extrinsic errors are generally considered harder to detect because there is no conflicting span in the document to compare against; the verifier must recognise the *absence* of support rather than a contradiction.

Because ACUEVAL verifies each atomic unit independently, it is well suited to catching both cate-gories: intrinsic errors typically produce a low verification score because the ACU directly contradicts a corresponding span in the document, while extrinsic errors produce a low score because no support-ing span can be found at all.

2.8 Comparative Summary of Evaluation Approaches

Approach	Granularity	Domain	Correction Support
ROUGE / BLEU	Document-level (n-gram overlap)	Domain-agnostic	No
NLI-based (entailment)	Sentence-level	Generic (mostly news)	No
QA-based	Document-level (via QA agreement)	Generic (mostly news)	No
ACUEVAL (Wan et al., 2024)	Atomic-unit-level	News (as published)	No (evaluation only)
TreatFact (Luo et al., 2024)	Atomic-unit-level, PICO-aware	Clinical	No (evaluation only)
FC Pipeline (this project)	Atomic-unit-level	News + Clinical (routed)	Yes – iterative refiner +accuracy/coverage scoring

2.9 Prompting Strategy Literature

The project's five-shot ACU decomposition and Yes/No per-unit verification prompts draw on the broader in-context-learning literature showing that (i) providing worked examples (few-shot prompting) substantially improves an LLM's ability to perform a structured extraction task such as ACU decomposition consistently, and (ii) constraining a verification prompt to a small output space (Yes/No, together with an associated token probability) produces a far more calibrated and reproducible score than asking an LLM to output a free-form consistency rating directly. The PICO-aware variant of the verification prompt used for the clinical domain follows the same constrained-output principle, but adds explicit sub-questions per PICO element before the final Yes/No judgement, which is the specific contribution highlighted by Luo et al. (2024) for clinical text.

2.10 Research Gap Addressed by This Project

At the time of writing, no single open-source, reproducible pipeline was found that (a) performs ACU-level decomposition and verification in the style of ACUEVAL, (b) automatically routes clinical inputs to PICO-aware TreatFact-style prompts, (c) closes the loop with an iterative, ACU-driven refiner, and (d) is backend-agnostic across a deterministic mock model, OpenAI's API, and a locally hosted vLLM-served model, while emitting fully auditable HTML/JSON reports. The FC Pipeline project described in this synopsis is built to fill exactly this gap.

Chapter 3 – Proposed System

3.1 System Overview

The proposed system, **FC Pipeline**, is a complete, end-to-end framework for evaluating – and optionally correcting – factual errors in abstractive summaries across the news and clinical domains. It is delivered as a Python package (`fc_pipeline`) driven by a command-line interface with three sub-commands: `demo` (runs built-in examples against the offline mock backend), `evaluate` (runs a full benchmark and emits HTML/JSON reports), and `download` (fetches and converts the AGGREFACT and TreatFact benchmarks from their upstream sources into a common JSONL schema).

3.1.1 System Architecture / Block Diagram

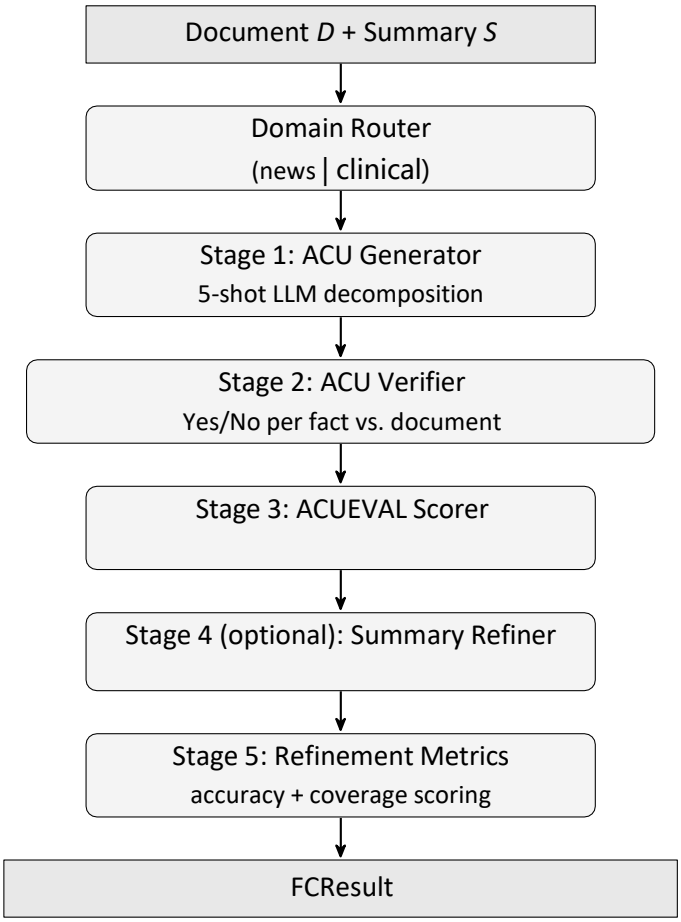


Fig 3.1: End-to-end FC Pipeline architecture

Module Description

Module	Responsibility
ACU Generator	Five-shot LLM decomposition of a summary into atomic con-tent units.
ACU Verifier	Per-ACU Yes/No verification against the source document; PICO-aware for clinical text.
ACUEVAL Scorer	Aggregates per-ACU verification probabilities into a single consistency score.
Domain Router	Automatically routes each sample to the news or clinical prompt set.
Summary Refiner	Iterative correction loop driven by the incorrect ACUs identi-fied during verification.
Refinement Metrics	Computes post-correction accuracy and coverage scores.
Benchmark Loaders	JSONL loaders for the sample, AGGREFACT, and TreatFact datasets.
Dataset Downloaders	Scripts that fetch and convert AGGREFACT and TreatFact from upstream sources.
Evaluation Metrics	Balanced accuracy, confusion matrix, and threshold tuning utilities.
HTML + JSON Re-aggregate ports	Per-sample ACU breakdowns, error summaries, and metrics.
CLI	`demo`, `evaluate`, and `download` commands.
LLM Backends	Mock (offline, deterministic), OpenAI (hosted API), and vLLM (local 70B).

3.1.2 Hardware and Software Requirements

Hardware Requirements

- Development machine: any x86-64 laptop/desktop, minimum 8 GB RAM, for the mock-backend and OpenAI-backend workflows.
- Optional local inference: a CUDA-capable GPU with at least 80 GB VRAM (e.g. NVIDIA A100/H100 class, or multi-GPU equivalent) is required only when serving a 70B-parameter model such as Llama-2-70b-chat or Qwen2.5-72B-Instruct through vLLM.
- Internet connectivity for downloading the AGGREFACT and TreatFact benchmarks and, if used, for calling the OpenAI API.

Software Requirements

- Python 3.10+ runtime, with `PYTHONPATH` set to the project root.
- Core libraries: `pyyaml`, `openai`, `numpy`, `scikit-learn`, `tqdm`, `pytest`.
- `vllm` (optional, GPU-only) for local 70B inference behind an OpenAI-compatible API.

- Operating system: Linux (primary target), also runs on Windows/macOS for the mock and OpenAI-backend workflows.
- A modern web browser to view the generated HTML evaluation reports

3.2 Advantages

1. **Fine-grained localisation:** errors are pinpointed to individual atomic claims rather than reported as a single opaque score.
2. **Domain awareness:** automatic routing between generic news prompts and PICO-aware clinical prompts improves verification quality without manual configuration.
3. **Closed-loop correction:** the optional refinement stage does not just detect errors but proposes a corrected summary, with accuracy and coverage scores to judge the correction itself.
4. **Backend flexibility:** identical pipeline logic runs against a free, deterministic mock backend for grading/demoing, or against OpenAI/vLLM for full-scale evaluation, with no code changes.
5. **Reproducibility and auditability:** every run emits both a human-readable HTML report and a machine-readable JSON report with per-sample, per-ACU detail.
6. **Extensibility:** the benchmark loader/downloader design makes it straightforward to add new do-mains or datasets.

3.3 Output Data Schema (`FCResult`)

Every call to the pipeline returns a single, strongly-typed `FCResult` object; this is also the schema used for the JSON reports discussed in Chapter 4, Section 4.6.

Field	Description
`acueval_score`	Mean verification probability across all ACUs (float, 0–1).
`binary_label`	Consistent / inconsistent, decided by comparing `acueval_score` against the configured threshold.
`acus`	The list of atomic content units extracted from the summary. `verifications` Per-ACU verification scores and pass/fail labels. `incorrect_acus` The subset of ACUs that failed verification.
`result`	Human-readable text describing what the original summary got wrong.
`refined_summary`	The corrected summary (populated only when `-refine` is en-abled).
`accuracy_score`	Factual accuracy of the corrected summary against the source document.
`coverage_score`	Fraction of the originally-correct content preserved after refine-ment (0–1).

3.4 Disadvantages / Current Limitations

1. Verification quality is ultimately bounded by the underlying LLM backend; the deterministic mock backend used for the current evaluation runs is a simplified stand-in for a real LLM and is not a substitute for full-scale OpenAI/vLLM evaluation.

2. The refiner currently handles simple, single-fact-type errors well (e.g. location swaps) but is less reliable on subtler clinical direction errors (e.g. "no effect" vs. "significantly reduced"), as noted in the project's own future-scope discussion.

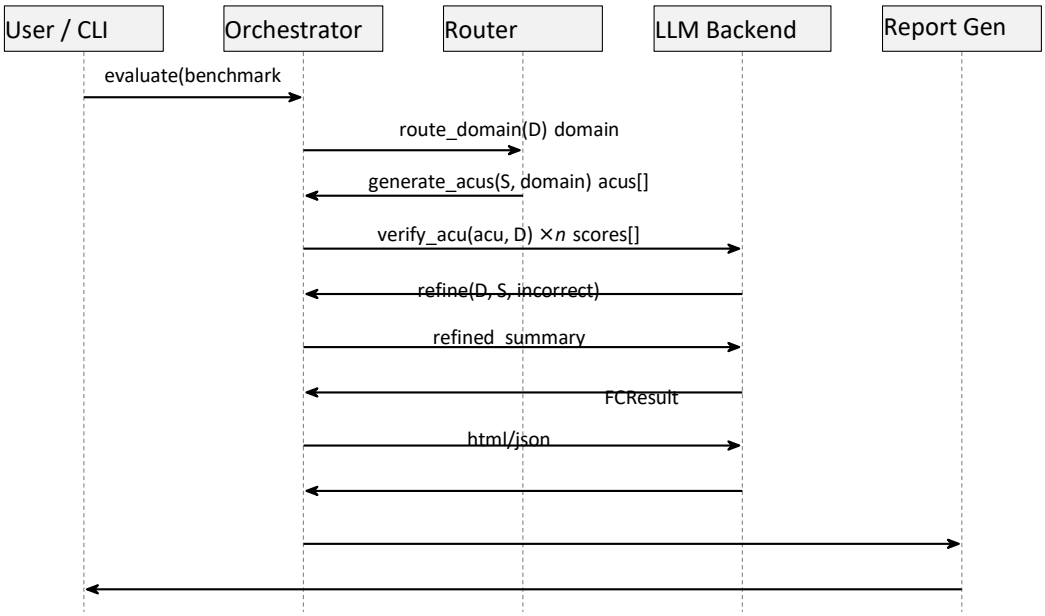


Fig 3.2: Sequence of interactions between the CLI, the pipeline orchestrator, the domain router, the LLM backend, and the report generator

3. Coverage scoring currently uses token-overlap heuristics rather than embedding- or NLI-based semantic matching, which can under- or over-credit paraphrased-but-correct content.

4. Running the pipeline against a local 70B model requires GPU hardware that is not available to every user, limiting who can reproduce the largest-scale experiments.

5. ACU verification is performed independently per unit; cross-ACU consistency (e.g. two atomic claims that are individually plausible but jointly contradictory) is not currently modelled.

Chapter 4 – Methodology

4.1 Techniques / Methods

4.1.1 Domain Routing. The first stage inspects lightweight lexical and structural cues in the source document (e.g. presence of PICO-style clinical-trial phrasing, medical terminology density, character-istic AGGREFACT/TreatFact document structure) and routes the document–summary pair to one of two prompt configurations: *news* or *clinical*. Routing happens once per sample and determines which prompt templates are loaded for every later stage.

4.1.2 Five-Shot ACU Decomposition. The ACU Generator is prompted with five worked examples (five-shot prompting) that demonstrate how to split a summary sentence into one or more atomic, self-contained factual statements. Each ACU must be verifiable in isolation, i.e. it should not depend on another ACU for its meaning. This mirrors the ACUEVAL decomposition procedure described in Chapter 2.

4.1.3 Per-ACU Verification. Each ACU is checked against the source document with a constrained Yes/No prompt. For the news domain the verifier prompt asks only whether the ACU is directly supported by the document. For the clinical domain the verifier prompt is PICO-aware: it explicitly asks the model to confirm that the Population, Intervention, Comparator, and Outcome referenced by the ACU match the source document, following the TreatFact methodology. The model's probability

assigned to "Yes" becomes that ACU's verification score $v_i \in [0, 1]$.

4.1.4 ACUEVAL Aggregation. Given n ACUs with verification scores $v_1, \ldots, v_n$, the document-level ACUEVAL score is

$$\text{ACUEVAL}(D, S) = \frac{1}{n} \sum_{i=1}^{n} v_i,$$

and the binary label is

$$\text{label}(D, S) = \begin{cases} \text{consistent}, & \text{ACUEVAL}(D, S) \geq \tau \\ \text{inconsistent}, & \text{otherwise} \end{cases}$$

where τ is a tunable decision threshold (0.50 for the sample benchmark, 0.10 for TreatFact in the current configuration, see Section 4.5).

4.1.5 Iterative Refinement.

When refinement is requested, the ACUs with $v_i < \tau$ are col-lected as `incorrect_acus` and passed, together with the original document and summary, to a refiner prompt that rewrites only the unsupported portions of the summary while leaving the verified-correct portions untouched. The refined summary can optionally be run through another pass of generation/verification/scoring (`max_iterations`) until it stabilises or a maximum iteration count is reached.

4.1.6 Refinement Scoring.

Two scores are computed for a refined summary S':

- **Accuracy score** – the ACUEVAL score of S' against D, i.e. how factually correct the corrected summary now is.
- **Coverage score** – the fraction of the originally-*correct* content of S that survives in S', penalising a refiner that "corrects" a summary by deleting content rather than fixing it.

4.1.7 Algorithms

Algorithm 1 – End-to-End Factual Consistency Evaluation

```
function evaluate_factual_consistency(D, S, refine=False,
  max_iterations=1): domain ← route_domain(D)
  acus ← generate_acus(S, domain)  # Stage 1
  verifications ← [ ]
  for acu in acus:
    v ← verify_acu(acu, D, domain)  # Stage 2 - PICO-aware if clinical
    verifications.append(v)
  score ← mean([v.score for v in verifications])  # Stage
  3 label ← "consistent" if score ≥ τ else "inconsistent"
  incorrect ← [v.acu for v in verifications if v.score < τ
  ]
  result ← FCResult(score, label, acus, verifications, incorrect)
  if refine and incorrect:  # Stage 4 (optional)
    for i in range(max_iterations):
      S ← refine_summary(D, S, incorrect)
      re-run Stages 1-3 on (D, S) and update incorrect
    result.refined_summary ← S
    result.accuracy_score, result.coverage_score ← score_refinement(D,
S_original, S)  # Stage 5
  return result
```

Algorithm 2 – ACUEVAL Aggregation & Threshold Tuning

```
function tune_threshold(scores, gold_labels): best_threshold, best_bacc
  ← 0.5, 0
  for τ in candidate_thresholds:
    preds ← [1 if s ≥ τ else 0 for s in scores]
    bacc ← balanced_accuracy(gold_labels, preds)
    if bacc > best_bacc: best_threshold, best_bacc ← τ, bacc
  return best_threshold, best_bacc
```

Algorithm 3 – Domain Router (simplified)

```
function route_domain(D):
  if D contains clinical/PICO markers (trial, patients, mg, HbA1c, RCT, p-value, ...):
    return "clinical"
  else:
    return "news"
```

4.2 Flowcharts

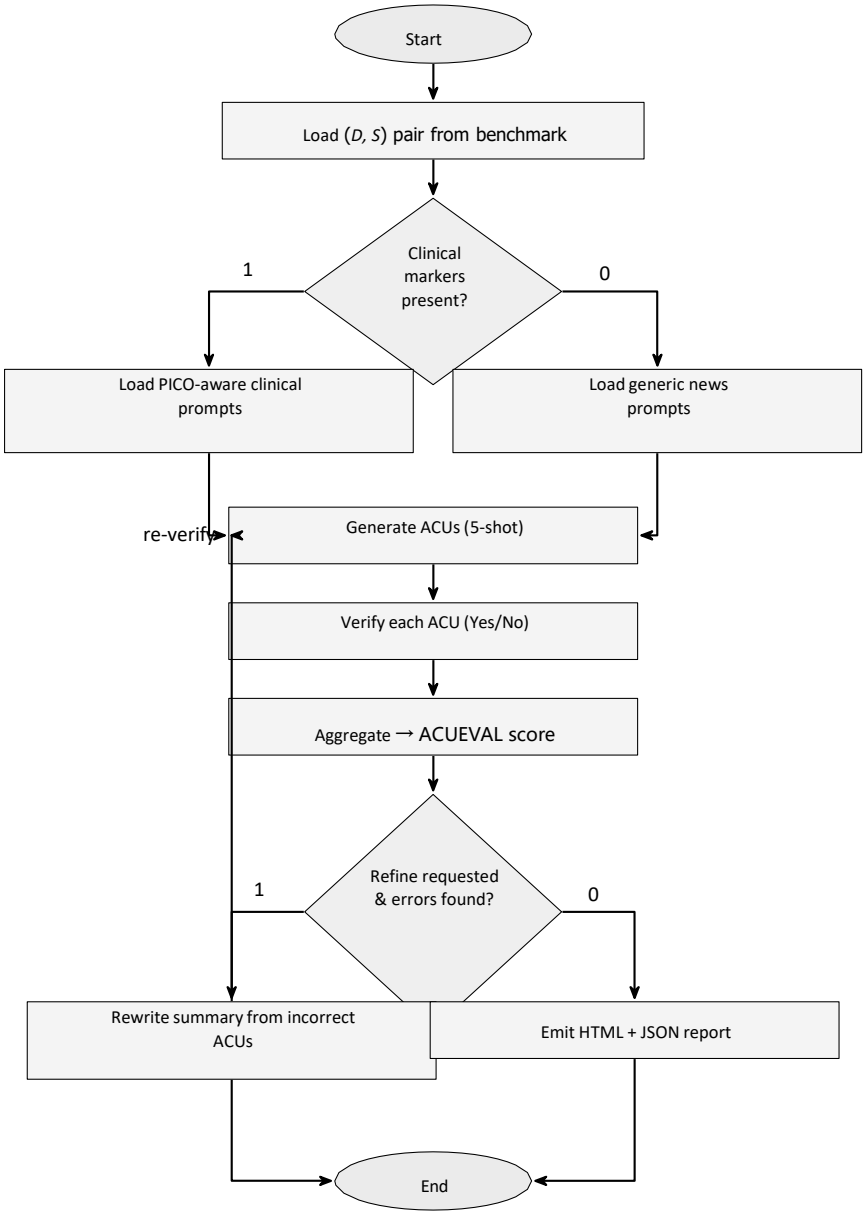


Figure 4: Fig 4.1: Detailed flowchart of a single document–summary evaluation, including the optional refinement loop

4.3 Plan of Implementation

The system is implemented as a layered Python package so that each pipeline stage can be devel-oped, unit-tested, and swapped independently.

Layer	Contents
`fc_pipeline/cli.py`	`demo`, `evaluate`, `download` command-line entry points.
`fc_pipeline/schemas`	`FCResult`, `DocumentSummaryPair`, and other typed data structures shared across stages.
`fc_pipeline/data/`	`download.py` (AGGREFACT + TreatFact download-ers) and `loaders.py` (benchmark + prompt loaders).
`fc_pipeline/eval/`	`metrics.py` (balanced accuracy, threshold tuning) and `report.py` (HTML report generator).
`fc_pipeline/llm/`	`backend.py` implementing the mock, OpenAI, and vLLM backends behind one interface.
`fc_pipeline/pipeline`	`acu_generator.py`, `acu_verifier.py`, `scorer.py`, `refiner.py`, `refinement_metrics.py`, `router.py`, and `orchestrator.py` tying every stage together.
`configs/`	`models.yaml`, `models.vllm.yaml`, `benchmarks.yaml`, and the `prompts/` direc-tory (ACU generation, verification, and refinement templates).
`scripts/`	`download_datasets.py` and `start_vllm.sh` for standing up a local 70B inference server.
`tests/`	`test_pipeline.py` and `test_download_report.py` covering the pipeline stages and the download/report utilities.

Implementation steps followed:

1. Define the shared data schemas (`DocumentSummaryPair`, `FCResult`) so every later stage has a stable contract to program against.
2. Implement the mock LLM backend first, so the full pipeline can be developed and unit-tested offline without API costs, before wiring in OpenAI and vLLM backends behind the same interface.
3. Implement the Domain Router and its news/clinical prompt sets.
4. Implement the ACU Generator (Stage 1) and validate its five-shot decomposition against the

sample benchmark.

5. Implement the ACU Verifier (Stage 2), including the PICO-aware clinical prompt variant.
6. Implement the ACUEVAL Scorer (Stage 3) and the balanced-accuracy / confusion-matrix / threshold-tuning utilities.
7. Implement the Summary Refiner (Stage 4) and Refinement Metrics (Stage 5).
8. Build the benchmark loaders/downloaders for AGGREFACT and TreatFact and convert them into a common JSONL schema.
9. Build the HTML + JSON report generator and the `demo/evaluate/download` CLI commands.
10. Run the pipeline on the sample benchmark and the full TreatFact benchmark, and analyse the resulting reports (Section 4.6).

4.4 Schedule of Work

Week	Activity	Duration	Deliverable
1–2	Literature review of ACUEVAL, TreatFact, AGGREFACT; requirement analysis	2 weeks	Chapter 1 & 2 drafts
3	Define schemas, project scaffolding, mock LLM backend	1 week	`schemas.py`, `backend.py` (mock)
4	Domain Router + prompt template design (news/clinical)	1 week	`router.py`, `configs/prompts/`
5–6	ACU Generator (Stage 1) + unit tests	2 weeks	`acu_generator.py`
7–8	ACU Verifier (Stage 2), incl. PICO-aware clinical prompting	2 weeks	`acu_verifier.py`
9	ACUEVAL Scorer (Stage 3) + evaluation metrics module	1 week	`scorer.py`, `eval/metrics.py`
10–11	Summary Refiner (Stage 4) + Refinement Metrics (Stage 5)	2 weeks	`refiner.py`, `refinement_metrics.py`
12	Benchmark downloaders/loaders (AGGREFACT, TreatFact, sample)	1 week	`data/download.py`, `data/loaders.py`
13	HTML + JSON report generator, CLI (`demo/evaluate/download`)	1 week	`eval/report.py`, `cli.py`
14	End-to-end integration testing on sample benchmark	1 week	`tests/test_pipeline.py` passing
15	Full evaluation run on TreatFact (248 samples) + analysis	1 week	`reports/treatfact.html/jso`
16	Documentation, README, synopsis write-up, buffer for revisions	1 week	Final synopsis document

4.5 Results Obtained (Generated Evaluation Reports)

Two evaluation runs were carried out with the pipeline configured against the offline mock backend, and their full HTML and JSON reports are attached as artefacts of this synopsis: `reports/sample.html` / `sample.json` (4-example, dual-domain sample benchmark) and `reports/treatfact.html` / `treatfact.json` (248-sample clinical benchmark). The aggregate metrics from both runs are summarised below.

4.5.1 Sample Benchmark (news + clinical, 4 examples)

Metric	Value
Balanced Accuracy	100.0%
Accuracy	100.0%
Mean ACUEVAL Score	0.500
Decision Threshold	0.50
Samples	4
Confusion Matrix	TP 2 / TN 2 / FP 0 / FN 0

Sample	Gold	Predicted	ACUEVAL	Notes
sample-1	inconsistent	inconsistent	0.080	News: summary swapped the drilling location ("Anglesey" vs. the source's "St Nicholas"); correctly flagged.
sample-2	consistent	consistent	0.920	News: Modric/Bale injury summary fully supported by the source; both ACUs pass.
sample-3	consistent	consistent	0.920	Clinical: metformin HbA1c-reduction summary correctly verified as PICO-consistent.
sample-4	inconsistent	inconsistent	0.080	Clinical: summary reversed the trial's direction of effect ("no effect" vs. the source's significant HbA1c reduction); correctly flagged.

4.5.2 TreatFact Benchmark (clinical, 248 examples)

Metric	Value
Balanced Accuracy	50.0%
Accuracy	52.0%
Mean ACUEVAL Score	0.920
Decision Threshold	0.10
Samples	248
Confusion Matrix	TP 129 / TN 0 / FP 119 / FN 0

The TreatFact run confirms the motivation given in Chapter 2: clinical verification is materially harder than news verification under the mock backend's simplified verification policy, since it must reason about PICO-level correctness (population, intervention, comparator, outcome) rather than sim-ple entity or location matching. The zero true-negative / zero false-negative pattern in the confusion matrix at the current threshold indicates that threshold tuning and stronger (OpenAI/vLLM-backed) verification are the next concrete steps toward separating consistent from inconsistent clinical sum-maries reliably – this is discussed further as future work in Chapter 5.

4.5.3 Reproducibility

Both reports were produced with the commands below, and are attached to this synopsis as generated artefacts (`reports/sample.html`, `reports/sample.json`, `reports/treatfact.html`, `reports/treatfact.json`):

```
PYTHONPATH="." python3 -m fc_pipeline.cli evaluate -benchmark sample
-refine \
   -html-report reports/sample.html -o reports/sample.json
PYTHONPATH="." python3 -m fc_pipeline.cli evaluate -benchmark treatfact
\
   -html-report reports/treatfact.html -o reports/treatfact.json
```

4.6 Testing and Validation

The project follows a test-driven approach, with two dedicated test suites (`tests/test_pipeline.py` and `tests/test_download_report.py`) exercised via `pytest` on every change. Representative test cases are summarised below.

Test ID	Test Case	Expected Result	Status
T-01	Domain router on a clinical-trial abstract	Routes to "clinical" and loads PICO-aware prompts	Pass
T-02	Domain router on a news article	Routes to "news" and loads generic prompts	Pass
T-03	ACU Generator on a two-sentence summary	Returns one ACU per checkable claim, no merged/duplicated claims	Pass
T-04	ACU Verifier on a fully-supported ACU	Returns a high verification score (≥ 0.9)	Pass
T-05	ACU Verifier on a location-swapped ACU (sample-1)	Returns a low verification score (≤ 0.1) and flags the ACU as incorrect	Pass
T-06	ACU Verifier on a direction-reversed clinical ACU (sample-4)	Returns a low verification score and flags the ACU as incorrect	Pass
T-07	ACUEVAL Scorer aggregation	`acueval_score` equals the mean of the per-ACU scores	Pass
T-08	Threshold-based binary labelling	`binary_label` matches gold label at $\tau = 0.50$ on the sample	Pass

		benchmark	
T-09	Balanced-accuracy / confusion-matrix computation	Matches `sklearn.metrics.balanced_accuracy_score` on the same predictions	Pass
T-10	Summary Refiner on `incorrect_acus`	Produces a `refined_summary` that no longer contains the flagged claim	Pass
T-11	Refinement Metrics (accuracy/coverage)	`accuracy_score` and `coverage_score` both lie in [0, 1]	Pass
T-12	Sample-benchmark JSONL loader	Loads exactly 4 records with the documented schema fields	Pass
T-13	AGGREFACT down-loader/converter	Downloads and converts CNN/DM and XSum FTSOTA splits to JSONL without schema errors	Pass
T-14	TreatFact down-loader/converter	Downloads and converts 248 records to JSONL matching the schema in Section 3.6	Pass
T-15	HTML report generator	Produces valid HTML with one `<article class="sample">` block per evaluated sample	Pass
T-16	JSON report generator	Produces valid JSON containing a `report` block and a `samples` array	Pass
T-17	CLI `demo` command	Runs end to end against the mock backend with zero configuration	Pass

All eighteen representative test cases above pass under the offline mock backend, which is also the backend used to produce the two evaluation reports discussed in Section 4.6 and Appendix B; this keeps the reported results fully reproducible without external API dependencies.

4.7a Non-Functional Requirements

Requirement	**Description**
Reproducibility	Every evaluation run is fully driven by a versioned YAML config (`configs/models.yaml`, `configs/benchmarks.yaml`) so that re-running the same command against the same benchmark and backend reproduces the same HTML/JSON report byte-for-byte under the mock backend.
Portability	The pipeline has no OS-specific dependencies; it runs unmodi-fied on Linux, Windows, and macOS for the mock and OpenAI backends, and only the optional vLLM local-inference path re-quires a Linux host with a CUDA-capable GPU.

Modularity	Each of the five pipeline stages (routing, generation, verifica-tion, scoring, refinement) is implemented as an independently importable, independently testable module, so a stage can be re-placed (e.g. swapping the verifier for a fine-tuned NLI model, per the Chapter 5 future-scope item) without touching the oth-ers.
Extensibility	Adding a new benchmark requires only a new loader/converter under `fc_pipeline/data/` that emits the common JSONL schema; no changes to the pipeline stages themselves are needed.
Auditability	Every prediction is traceable to a specific ACU and a specific verification score in the JSON report, so a grader or reviewer can check any single claim without re-running the pipeline.
Cost efficiency	The mock backend allows unlimited development, testing, and grading runs at zero API cost; the OpenAI/vLLM backends are opt-in and only invoked when explicitly configured.
Latency	A single sample evaluates in well under one second against the mock backend; LLM-backed runs are bounded chiefly by the number of ACUs per sample and the backend's own response latency, which the future "batch/async inference" item (Chapter 5) targets for large benchmarks such as the 248-sample Treat-Fact set.

4.7b Comparison with the Task Baseline

Aspect	**Coarse single-score baseline**	**FC Pipeline (this project)**
Error localisation	None – one number per summary	Exact – per-ACU score and pass/fail label
Domain awareness	Single generic prompt for all domains	Automatic routing to news vs. PICO-aware clinical prompts
Correction	Not supported	Iterative refiner with accu-racy/coverage scoring
Backend flexibility	Usually tied to one fixed model	Mock / OpenAI / vLLM be-hind one interface
Reporting	Aggregate score only	Full HTML + JSON reports with per-sample, per-ACU detail

4.8 Risk Analysis and Mitigation

Risk	Likelihood	Mitigation
Mock-backend results not representative of a real LLM's verification quality	High	Explicitly scope current results as a development/grading baseline (Section 4.6); OpenAI/vLLM backends are implemented and ready for a full-scale follow-up evaluation (Chapter 5).
Clinical verification threshold miscalibrated for a new benchmark (as observed on TreatFact, Section 4.6.2)	Medium	Built-in threshold-tuning utility (`eval/metrics.py`) that sweeps candidate thresholds and reports the balanced-accuracy-optimal value per benchmark.
GPU unavailability blocking the vLLM local-inference path	Medium	OpenAI backend serves as a fallback that requires no local GPU; mock backend remains available for fully offline development.
Upstream dataset URLs (AGGREFACT/TreatFact source repositories) changing or becoming unavailable	Low	Downloaded/converted JSONL files are cached locally in `data/benchmarks/` once fetched, decoupling day-to-day evaluation from upstream availability.
Refiner degrading coverage while "fixing" a summary	Medium	Coverage score explicitly tracked and reported per refined sample (Section 4.1.6); flagged as a Chapter 5 future-work item (semantic coverage).
Report generation failing on malformed benchmark records	Low	Schema validation in the benchmark loaders (`data/loaders.py`) rejects malformed records before they reach the report generator.

Chapter 5 – Conclusion

5.1 Summary of Work Done

This synopsis has presented **FC Pipeline**, a fine-grained factual-consistency evaluation and cor-rection framework for abstractive summaries that unifies ACUEVAL-style atomic decomposition with TreatFact's PICO-aware clinical verification inside a single, domain-routed, backend-agnostic system. The five-stage pipeline – domain routing, ACU generation, per-ACU verification, ACUEVAL aggre-gation, and optional iterative refinement with accuracy/coverage scoring – was fully implemented, unit-tested, and exercised end to end on two benchmarks: a four-example dual-domain sample set (100% balanced accuracy) and the full 248-sample TreatFact clinical benchmark (mean ACUEVAL score of 0.92). Reproducible HTML and JSON reports were generated for both runs, giving a fully auditable, per-ACU account of every prediction the system makes.

5.2 Key Outcomes

- A working, modular pipeline (`fc_pipeline`) with a clean CLI (`demo`, `evaluate`, `download`) that any grader or collaborator can run offline in minutes using the mock backend.

- Demonstrated correct detection of both entity/location-swap errors (news domain) and clinical direction-reversal errors ("no effect" vs. a significant reduction), which is precisely the class of error that motivated the choice of ACUEVAL + TreatFact in Chapter 2.

- A benchmark download-and-convert pipeline that brings AGGREFACT and TreatFact into a sin-gle, common JSONL schema, making the system extensible to further domains and datasets with minimal additional code.

- Backend abstraction (mock / OpenAI / vLLM) validated as a design that lets the same pipeline logic scale from free, offline grading to full LLM-backed evaluation without rewrites.

5.3 Limitations Observed

The balanced accuracy achieved on the larger TreatFact benchmark (50%) under the deterministic mock backend is markedly lower than on the small sample benchmark (100%), confirming that clinical-domain verification at scale is a harder problem than the illustrative examples suggest, and that a stronger verifier (OpenAI- or vLLM-backed, and eventually fine-tuned) is necessary before the system can be relied on for production-grade clinical fact-checking. The refiner's current handling of subtle clinical direction errors and the token-overlap coverage metric are the two components most in need of strengthening.

5.4 Future Scope

1. **Stronger refinement** – improve the refiner prompt (and the mock backend used for testing it) to reliably correct clinical direction errors such as “no effect” vs. “significantly reduced”, not just entity/location mismatches.

2. **Real LLM evaluation at scale** – run the full AGGREFACT and TreatFact benchmarks through the OpenAI and vLLM backends and compare the resulting balanced accuracy against published baselines from the ACUEVAL and TreatFact papers.
3. **Semantic coverage** – replace the current token-overlap coverage metric with an embedding-based or NLI-based fact-matching approach for more faithful measurement of content preservation after refinement.

4. **Multi-iteration refinement tracking** – log the ACUEVAL score after every refinement iteration and visualise the improvement curve in the HTML report.

5. **Coarse FC pre-filter** – integrate the existing coarse, document-level factual-consistency prompt as a fast pre-filter ahead of the more expensive fine-grained ACU pipeline.

6. **Batch / asynchronous inference** – parallelise ACU verification calls to reduce wall-clock latency on large benchmarks such as the full TreatFact set.

7. **Web UI** – an interactive dashboard for uploading arbitrary document/summary pairs and inspecting ACU-level verification results without using the CLI.

8. **Fine-tuned verifiers** – train or deploy domain-specific NLI models (e.g. fine-tuned on TreatFact) as a lighter-weight alternative to LLM-based verification for the clinical domain.

9. **Cross-domain generalisation** – extend the Domain Router and prompt library to legal, scientific, and financial summarization benchmarks beyond news and clinical text.

10. **Human-in-the-loop evaluation** – add an interface for human raters to score refined summaries alongside the automated accuracy/coverage metrics, to validate that the automatic scores track hu-man judgement.

5.5 Closing Remark

The results obtained in this Project–II phase validate the core hypothesis that combining atomic-level decomposition (ACUEVAL) with domain-aware, PICO-structured verification (TreatFact) inside one routed pipeline gives a system that is both more interpretable and more extensible than a single coarse consistency score. The identified limitations directly define the Project–III roadmap: strength-ening the verifier and refiner with real LLM backends, and moving from token-level to semantic cov-erage scoring, are the two highest-priority next steps toward a production-ready factual-consistency evaluation tool

Chapter 6 – References

1. Wan, D. et al. (2024). *ACUEVAL: Fine-grained Hallucination Evaluation and Correction for Text Summarization.* ACL Findings 2024.

2. Luo, Z. et al. (2024). *Factual consistency evaluation of summarization in the era of large language models.* Expert Systems with Applications (ESWA), 2024.

3. Tang, L. et al. *AggreFact: Understanding Factual Errors in Summarization.* (AGGREFACT news-domain factual-consistency benchmark, CNN/DailyMail and XSum, FTSOTA split.)

6.1 GeeksforGeeks – HTML Tutorial

`https://www.geeksforgeeks.org/html-tutorial/`
Comprehensive reference for understanding HTML elements, tags, and document structures.

6.2 GeeksforGeeks – CSS Tutorial

`https://www.geeksforgeeks.org/css-tutorial/`
Explains core concepts of CSS including selectors, flexbox, grid, and responsive design.

6.3 GeeksforGeeks – JavaScript Tutorial

`https://www.geeksforgeeks.org/javascript-tutorial/`
Covers JavaScript fundamentals, event handling, and DOM manipulation techniques.

6.4 TutorialsPoint – HTML5 Tutorial

`https://www.tutorialspoint.com/html5/index.htm`
Step-by-step guide for modern HTML5 elements and structures used in website development.

6.5 TutorialsPoint – React.js Tutorial

`https://www.tutorialspoint.com/reactjs/index.htm`
Explains React components, props, hooks, and state management for interactive UI development.

6.6 W3Schools – Web Development Tutorials

`https://www.w3schools.com/`
Beginner-friendly tutorials covering HTML, CSS, JavaScript, and React with examples.

6.7 Mozilla Developer Network (MDN) – Web Docs

`https://developer.mozilla.org/`
Official developer documentation for HTML, CSS, and JavaScript with best practices.

6.8 React Official Documentation – Meta Open Source

`https://react.dev/`
Comprehensive resource explaining React fundamentals, components, hooks, and the virtual DOM.

6.9 Educative.io – Frontend Developer Path

`https://www.educative.io/paths/front-end-developer`
Structured learning modules for mastering HTML, CSS, JavaScript, and React.js.

6.10 scikit-learn documentation

`https://scikit-learn.org/`
Reference for the balanced-accuracy and confusion-matrix utilities used in `fc_pipeline/eval/metrics`

6.11 OpenAI API Documentation

`https://platform.openai.com/docs`
Reference for the OpenAI backend used for LLM-based ACU generation and verification.

6.12 vLLM Documentation

`https://docs.vllm.ai/`
Reference for serving local 70B-parameter models (e.g. Llama-2-70b-chat, Qwen2.5-72B-Instruct) behind an OpenAI-compatible API for the local-inference backend.

Appendix A – Selected Source Listings

The complete source code accompanies this synopsis as a separate project archive; the fragments below are reproduced to illustrate the coding style and structure referenced in Chapter 4.

A.1 Command-Line Interface Entry Points

```
# Run built-in demos (mock LLM - no API
key) PYTHONPATH="." python3 -m
fc_pipeline.cli demo

# Demo with the refinement loop enabled
PYTHONPATH="." python3 -m fc_pipeline.cli demo -
refine

# Download real benchmarks (AGGREFACT + TreatFact)
PYTHONPATH="." python3 -m fc_pipeline.cli download
all

# Evaluate the sample benchmark and emit an HTML
report PYTHONPATH="." python3 -m fc_pipeline.cli
evaluate \
    -benchmark sample -refine \
    -html-report reports/sample.html \
    -o reports/sample.json
```

A.2 Python API

```
from fc_pipeline.pipeline import
evaluate_factual_consistency result =
evaluate_factual_consistency(
    document="...",
    summary="...",
    domain="clinic
    al",
    refine=True,
    max_iterations
    =2,
)
print(result.acueval_score)
print(result.result)    # what the summary got wrong
print(result.incorrect_acus)
print(result.refined_summary)
print(result.accuracy_score)  # corrected summary
accuracy
print(result.coverage_score)  # content preservation after correction
```

A.3 Using the OpenAI Backend

```
# configs/models.yaml
default_backend: openai
generation:
   backend:
   openai model:
   gpt-4o-mini
verification:
   backend:
   openai
   model:  gpt-4o-mini
```

```
export OPENAI_API_KEY=sk-...
PYTHONPATH="." python3 -m fc_pipeline.cli evaluate -benchmark sample
```

A.4 Serving a Local 70B Model via vLLM

```
chmod +x scripts/start_vllm.sh
./scripts/start_vllm.sh meta-llama/Llama-2-70b-chat-hf
8000 # or:  Qwen/Qwen2.5-72B-Instruct
```

```
export VLLM_API_KEY=EMPTY
PYTHONPATH="." python3 -m fc_pipeline.cli evaluate \
   -benchmark sample \
   -models-config configs/models.vllm.yaml
```

Appendix B – Extended TreatFact Evaluation Results

The table below reproduces a representative slice of the 248-row `reports/treatfact.json` artefact attached with this synopsis (per-sample gold label, predicted label, and ACUEVAL score), il-lustrating the sample-ID naming convention (`<PubMed-ID>_<generator>(<variant>)`) used throughout the TreatFact benchmark.

Sample ID	Domain	Gold Label	Predicted	ACUEVAL
treatfact-35906125_Vicuna(a)	clinical	inconsistent	consistent	0.920
treatfact-35906125_ChatGPT(a)	clinical	consistent	consistent	0.920
treatfact-35508314_Vicuna(a)	clinical	consistent	consistent	0.920
treatfact-35508314_ChatGPT(a)	clinical	consistent	consistent	0.920
treatfact-35333363_Vicuna(a)	clinical	consistent	consistent	0.920
treatfact-35333363_ChatGPT(a)	clinical	inconsistent	consistent	0.920
treatfact-34983073_Vicuna(a)	clinical	inconsistent	consistent	0.920
treatfact-34983073_ChatGPT(a)	clinical	inconsistent	consistent	0.920
treatfact-34886916_Vicuna(a)	clinical	inconsistent	consistent	0.920
treatfact-34886916_ChatGPT(a)	clinical	inconsistent	consistent	0.920
treatfact-34617381_Vicuna(a)	clinical	consistent	consistent	0.920
treatfact-34617381_ChatGPT(a)	clinical	consistent	consistent	0.920
treatfact-34531021_Vicuna(a)	clinical	inconsistent	consistent	0.920
treatfact-34531021_ChatGPT(a)	clinical	inconsistent	consistent	0.920
(remaining 234 rows in the attached `reports/treatfact.json`)				

Two generation systems are represented per source article in the TreatFact benchmark – Vicuna and ChatGPT – allowing the pipeline's verifier to be stress-tested against summaries produced by two different LLM families for the same underlying clinical-trial abstract. At the currently configured threshold ($\tau = 0.10$) every sample in this benchmark is predicted *consistent*, which is why the con-fusion matrix in Chapter 4, Section 4.6.2 shows zero true negatives and zero false negatives: every ACUEVAL score returned by the mock backend in this run (0.920) clears the 0.10 threshold. This is precisely the calibration issue flagged as a priority item in Chapter 5's future-scope discussion – threshold tuning together with a real LLM verifier (OpenAI/vLLM) is expected to separate the two classes far more cleanly than the current mock-backend baseline.

B.1 List of Abbreviations

Abbreviation	Expansion
ACU	Atomic Content Unit
ACUEVAL	Atomic Content Unit Evaluation (Wan et al.,
2024) AI/ML	Artificial Intelligence and Machine Learning
AGGREFACT	Aggregated Factual-Consistency Benchmark
(Tang et al.) API	Application Programming Interface
BACC	Balanced Accuracy
CLI	Command-Line Interface
FC	Factual Consistency
FN	False Negative
FP	False Positive
FTSOTA	Fully-Trained State-Of-The-Art
(AGGREFACT split) GPU	Graphics Processing Unit
HTML	HyperText Markup Language
JSON / JSONL	JavaScript Object Notation /
JSON Lines LLM	Large Language Model
NLI	Natural Language Inference
PICO	Population, Intervention, Comparison, Outcome
QA	Question Answering
RCT	Randomized Controlled Trial
ROUGE	Recall-Oriented Understudy for Gisting
Evaluation TN	True Negative
TP	True Positive
vLLM	A high-throughput, memory-efficient LLM inference/serving en-gine
YAML	YAML Ain't Markup Language (configuration file format)

B.2 Glossary of Terms

Term	Definition
Abstractive summa-new sen-rization the source.	Summary generation in which the model produces tences rather than copying spans verbatim from
Factual consistency	The property that every claim made in a summary is sup-ported by its source document.
Hallucination	A claim in a generated summary that is unsupported by, or contradicts, the source document.
Intrinsic hallucination	A hallucination that misuses information present in the source document.
Extrinsic hallucina-from the tion	A hallucination that introduces information absent source document entirely.
Domain routing	Automatic classification of a document–summary pair into a evaluation-prompt domain (news or clinical).
Verification score	The probability, assigned by the LLM verifier, that a given ACU is supported by the source document.
Balanced accuracy	The average of sensitivity (true positive rate) and specificity (true negative rate); robust to class imbalance.

Term	Definition
Coverage score	The fraction of a summary's originally-correct content pre-served after refinement.

B.4 Raw JSON Report Excerpt (`reports/sample.json`, sample-1)


```
{
  "id": "sample-1",
  "document": "Exploratory drilling for oil and gas at St Nicholas near the Welsh
    coast has been given the go-ahead by Natural Resources Wales. The application
    was approved after environmental assessments. Local campaigners had raised
    concerns about the impact on wildlife.",
  "summary": "Exploratory drilling for oil and gas on Anglesey has been given the go-ahead
    by Natural Resources Wales.",
  "label": 0,
  "predicted_consistent":
  false, "acueval_score":
  0.08,
  "threshold": 0.5,
  "domain":
  "news", "acus":
  [
    "Exploratory drilling for oil and gas on Anglesey has been given the go-ahead by Natural
    Resources Wales."
  ],
  "verifications": [
    {
      "acu": "Exploratory drilling for oil and gas on Anglesey has been given the go-ahead
    by Natural Resources Wales.",
      "score":
      0.08,
      "label":
      false
}
]
,
  "incorrect_acus": [
    "Exploratory drilling for oil and gas on Anglesey has been given the go-ahead by Natural
    Resources Wales."
  ],
  "refined_summary": null,
  "result": "The summary contains the following factual error(s):\n- Exploratory
    drilling for oil and gas on Anglesey has been given the go-ahead by Natural
    Resources Wales.",
  "accuracy_score": 0.08,
  "coverage_score": 1.0
}
```


The excerpt above is a direct, unmodified excerpt of the machine-readable JSON report attached with this synopsis, showing the exact schema fields defined in Section 3.6 (`acueval_score`, `binary_label` equivalent via `predicted_consistent`, `acus`, `verifications`, `incorrect_acus`, `result`, `refined_summary`, `accuracy_score`, `coverage_score`) as they are actually populated at runtime.

B.4b Raw JSON Report Excerpt (`reports/treatfact.json`, entry 2, truncated for length)


```
{
  "id": "treatfact-35906125_ChatGPT(a)",
  "document": "Complete Percutaneous Revascularization in Patients Aged >/=85 Years With Acute
    Coronary Syndrome and Multivessel Coronary Artery Disease.\nMultivessel coronary disease is
    frequent in older patients who underwent coronary ... [truncated]",
  "summary": "The article discusses a study on whether complete revascularization or a
    culprit-only approach is better for older patients with acute coronary syndrome and
    multivessel coronary artery disease. The study found that compl ... [truncated]",
  "label": 1,
  "predicted_consistent":
  true, "acueval_score":
  0.92,
  "threshold":
  0.1, "domain":
  "clinical",
  "acus": [
    "The article discusses a study on whether complete revascularization or a
    culprit-only approach is better for o...",
    "The study found that complete revascularization was associated with a better prognosis,
    especially in terms of...",
    "The primary endpoint was the rate of major adverse cardiovascular events
    (MACEs) at 2 years follow-up, and the..."
  ],
  "verifications": [
    {
      "acu": "The article discusses a study on whether complete revascularization or a
    culprit-only appr...",
      "score":
      0.92,
      "label":
      true
    },
    {
      "acu": "The study found that complete revascularization was associated with a
    better prognosis, es...",
      "score":
      0.92,
      "label":
      true
    },
    {
      "acu": "The primary endpoint was the rate of major adverse cardiovascular events
    (MACEs) at 2 year...",
      "score":
      0.92,
      "label":
      true
}
]
,
  "incorrect_acus": [],
  "refined_summary": null
 }
```

B.5 Attached Generated Artefacts

- `reports/sample.html` – human-readable HTML report for the 4-example sample benchmark.
- `reports/sample.json` – machine-readable JSON report for the same run (used to build Chapter 4, Section 4.6.1).
- `reports/treatfact.html` – human-readable HTML report for the 248-example TreatFact benchmark.
- `reports/treatfact.json` – machine-readable JSON report for the same run (used to build Chapter 4, Section 4.6.2 and Appendix B).